City of Light v3.0 — Warp Agent Implementation Guide

Architecture Pattern: Spirit as Brain, OpenFang as Skeletal/Immune System

---

Overview

This document provides step-by-step instructions for the Warp terminal agent to implement City of Light v3.0 on a Debian 13 (Trixie) host with the GEEKOM A5 hardware profile. The architecture uses **OpenFang as the runtime/security substrate** (skeletal and immune system) while implementing a **custom Active Inference "Spirit" agent as the brain** that orchestrates City operations.

The implementation follows **Scenario 2**: OpenFang provides scheduling, memory, security, and multi-agent coordination, while the Spirit implements Free Energy Principle-based planning and decision-making.

Prerequisites Verification

Before starting, verify the following on your machine:

- Debian 13 (Trixie) installed and accessible via SSH
- Hostname set to `wera-ss-pt-sn-1`
- Static IP configured on LAN
- Docker Engine and Docker Compose plugin installed
- Data disk mounted at `/data` with LUKS2 encryption
- Sudo access for the deployment user
- At least 32GB RAM and 500GB available storage

Verification commands:

Check OS version

cat /etc/os-release | grep VERSION

Check hostname

hostnamectl status | grep hostname

Check Docker

docker --version
docker compose version

Check data mount

df -h /data
sudo cryptsetup luksDump /dev/sda1 | head -n 20

Check available resources

free -h
df -h

Phase 1: Repository Structure Setup

Create the project directory structure that maps to the City of Light buildings metaphor.

Create base directory

mkdir -p ~/city-of-light
cd ~/city-of-light

Create core structure

mkdir -p
{ops,compose,prometheus,alertmanager,spirit,models,openfang}

Ops directory (runbooks and config)

mkdir -p ops/{backups,logs,certificates}

Prometheus directory (Library building)

mkdir -p prometheus/{rules,targets}

Alertmanager directory

mkdir -p alertmanager/templates

Spirit directory (the Brain)

mkdir -p spirit/{src,memory,logs,skills,hands}
mkdir -p spirit/memory/{embeddings,sessions,audit}

OpenFang directory (Skeletal/Immune system)

mkdir -p openfang/{hands,config,data}

Models directory (University building - LLM weights)

mkdir -p models/llama-cpp

Verify structure:

tree -L 2 ~/city-of-light

Phase 2: Configuration Files

2.1 Create ops/CONFIG.md

cat > ops/CONFIG.md << 'EOF'

City of Light Configuration

Created: $(date -I)

Machine: wera-ss-pt-sn-1

Network

LAN_IP=192.168.1.100/24
DOMAIN_MODE=A # A=LAN-only, B=Public domain
TLS_MODE=self-signed

Admin Accounts

NEXTCLOUD_ADMIN=admin
NEXTCLOUD_EMAIL=admin@wera.local
RUNDECK_ADMIN=operator
RUNDECK_EMAIL=operator@wera.local

Storage

DATA_MOUNT=/data
DOCKER_VOLUMES=/data/docker-volumes
MODEL_STORAGE=/data/models
BACKUP_LOCATION=/data/backups

Alert Channel

ALERT_CHANNEL=telegram
ALERT_DESTINATION=@city_of_light_alerts

OpenFang Configuration

OPENFANG_PORT=8080
OPENFANG_MEMORY_PATH=/data/openfang/memory.db

Spirit Configuration

SPIRIT_HEARTBEAT_INTERVAL=300 # 5 minutes
SPIRIT_LLM_ENDPOINT=http://localhost:8081
SPIRIT_PLANNING_HORIZON=6
EOF

2.2 Create compose/.env Template

cat > compose/.env.template << 'EOF'

Docker Compose Environment Variables

Copy to .env and fill in secrets

Nextcloud AIO

NEXTCLOUD_DATADIR=/data/nextcloud
APACHE_PORT=8080
APACHE_IP_BINDING=0.0.0.0

Rundeck

RUNDECK_GRAILS_URL=http://localhost:4440
RUNDECK_DATABASE_URL=jdbc:postgresql://postgres:5432/rundeck

Prometheus

PROMETHEUS_RETENTION=30d

Alertmanager

ALERTMANAGER_WEBHOOK_URL=http://openfang:8080/webhook/prometheus

llama.cpp

LLAMA_HOST=0.0.0.0
LLAMA_PORT=8081
LLAMA_THREADS=8
LLAMA_CTX_SIZE=4096

OpenFang

OPENFANG_LLM_PROVIDER=openai
OPENFANG_LLM_ENDPOINT=http://llama-cpp:8081/v1
OPENFANG_MEMORY_DB=/data/openfang/memory.db

Spirit

SPIRIT_NAME=city_spirit_001
SPIRIT_LOG_LEVEL=INFO
EOF

Important: Copy template to `.env` and add actual secrets:

cp compose/.env.template compose/.env
nano compose/.env # Add real passwords, tokens, API keys

Phase 3: Docker Compose Stack

3.1 Create Main Compose File

cat > compose/docker-compose.yml << 'EOF'
version: "3.8"

networks:
city_internal:
driver: bridge

volumes:
prometheus_data:
alertmanager_data:
rundeck_data:
postgres_data:
openfang_data:

services:

Library: Prometheus (metrics and knowledge)

prometheus:
image: prom/prometheus:latest
container_name: city-prometheus
networks:

- city_internal
ports:
- "9090:9090"
volumes:
- prometheus_data:/prometheus
- ../prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
- ../prometheus/rules:/etc/prometheus/rules:ro
command:
- '--config.file=/etc/prometheus/prometheus.yml'
- '--storage.tsdb.path=/prometheus'
- '--storage.tsdb.retention.time=${PROMETHEUS_RETENTION:-30d}'
restart: unless-stopped

Library: Alertmanager

alertmanager:
image: prom/alertmanager:latest
container_name: city-alertmanager
networks:
- city_internal
ports:
- "9093:9093"
volumes:
- alertmanager_data:/alertmanager
-
../alertmanager/alertmanager.yml:/etc/alertmanager/alertmanager.y
ml:ro
restart: unless-stopped

Library: Node Exporter (host metrics)

node-exporter:
image: prom/node-exporter:latest
container_name: city-node-exporter
networks:
- city_internal

pid: host
volumes:
- /proc:/host/proc:ro
- /sys:/host/sys:ro
- /:/rootfs:ro
command:
- '--path.procfs=/host/proc'
- '--path.sysfs=/host/sys'

```
  - '--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($$
restart: unless-stopped
```

Library: cAdvisor (container metrics)

cadvisor:
image: gcr.io/cadvisor/cadvisor:latest
container_name: city-cadvisor
networks:
- city_internal
volumes:
- /:/rootfs:ro
- /var/run:/var/run:ro
- /sys:/sys:ro
- /var/lib/docker/:/var/lib/docker:ro
restart: unless-stopped

Factory: PostgreSQL (Rundeck database)

postgres:
image: postgres:15-alpine
container_name: city-postgres
networks:
- city_internal

environment:
POSTGRES_DB: rundeck
POSTGRES_USER: rundeck
POSTGRES_PASSWORD: ${RUNDECK_DB_PASSWORD}
volumes:
- postgres_data:/var/lib/postgresql/data
restart: unless-stopped

Factory: Rundeck (automation and orchestration)

rundeck:
image: rundeck/rundeck:latest
container_name: city-rundeck
networks:
- city_internal
ports:
- "4440:4440"
environment:
RUNDECK_GRAILS_URL: ${RUNDECK_GRAILS_URL}
RUNDECK_DATABASE_DRIVER: org.postgresql.Driver
RUNDECK_DATABASE_URL: ${RUNDECK_DATABASE_URL}
RUNDECK_DATABASE_USERNAME: rundeck
RUNDECK_DATABASE_PASSWORD: ${RUNDECK_DB_PASSWORD}
volumes:
- rundeck_data:/home/rundeck/server/data
depends_on:
- postgres
restart: unless-stopped

University: llama.cpp (local LLM inference)

llama-cpp:
image: ghcr.io/ggerganov/llama.cpp:server
container_name: city-llama-cpp

networks:
- city_internal
ports:
- "8081:8081"
volumes:
- /data/models:/models:ro
environment:
- HOST=$LLAMA_HOST : -0.0.0.0 - PORT =$
{LLAMA_PORT:-8081}
command:
- "-m"
- "/models/qwen3-4b-q5_k_m.gguf"
- "--host"
- "0.0.0.0"
- "--port"
- "8081"
- "--threads"
- "$LLAMA_THREADS : -8" - - ctx - size" -$
{LLAMA_CTX_SIZE:-4096}"
- "--parallel"
- "4"
restart: unless-stopped

Skeletal/Immune System: OpenFang

openfang:
image: rightnow/openfang:latest
container_name: city-openfang
networks:
- city_internal
ports:
- "8080:8080"
volumes:
- openfang_data:/data
- ../openfang/hands:/hands:ro
- ../openfang/config:/config:ro
environment:

- OPENFANG_LLM_PROVIDER=
$OPENFANG_LLM_PROVIDER - OPENFANG_LLM_ENDP$
{OPENFANG_LLM_ENDPOINT}
- OPENFANG_MEMORY_DB=${OPENFANG_MEMORY_DB}
restart: unless-stopped

Brain: Spirit (Active Inference orchestrator)

spirit:
build:
context: ../spirit
dockerfile: Dockerfile
container_name: city-spirit
networks:
- city_internal
volumes:
- /data/spirit/memory:/app/memory
- /data/spirit/logs:/app/logs
- ../spirit/HEARTBEAT.md:/app/HEARTBEAT.md:ro
- ../spirit/RULES.md:/app/RULES.md:ro
- ../spirit/SKILLS.md:/app/SKILLS.md:ro
environment:
- SPIRIT_NAME=
$SPIRIT_NAME - SPIRIT_HEARTBEAT_INTERVAL =$
{SPIRIT_HEARTBEAT_INTERVAL}
- SPIRIT_LLM_ENDPOINT=
$SPIRIT_LLM_ENDPOINT - SPIRIT_PROMETHEUS_URL =$
{SPIRIT_LOG_LEVEL:-INFO}
depends_on:
- prometheus
- rundeck
- llama-cpp
- openfang
restart: unless-stopped

EOF

Phase 4: Prometheus Configuration (Library)

4.1 Create prometheus.yml

cat > prometheus/prometheus.yml << 'EOF'
global:
scrape_interval: 15s
evaluation_interval: 15s
external_labels:
city: 'wera-ss-pt-sn-1'
realm: 'digital_land'

alerting:
alertmanagers:
- static_configs:
- targets:
- alertmanager:9093

rule_files:

- '/etc/prometheus/rules/*.yml'

scrape_configs:

Host metrics (Real Land sensors)

- job_name: 'node-exporter'
  static_configs:
  - targets: ['node-exporter:9100']
    labels:
    building: 'real_land'
    type: 'infrastructure'

Container metrics (Digital Land buildings)

- job_name: 'cadvisor'
  static_configs:
    - targets: ['cadvisor:8080']
      labels:
      building: 'digital_land'
      type: 'containers'

Spirit heartbeat (Brain vitals)

- job_name: 'spirit'
  static_configs:
    - targets: ['spirit:8000']
      labels:
      building: 'spirit'
      type: 'agent'

OpenFang metrics (Skeletal/Immune system)

- job_name: 'openfang'
  static_configs:
    - targets: ['openfang:8080']
      labels:
      building: 'openfang'
      type: 'agent_os'
      EOF

4.2 Create Alert Rules

cat > prometheus/rules/city_alerts.yml << 'EOF'
groups:

- name: city_health
  interval: 30s
  rules:
    - alert: HighDiskUsage
      expr: (node_filesystem_avail_bytes{mountpoint="/data"} / node_filesystem_size_bytes{mountpoint="/data"}) < 0.1
      for: 5m
      labels:
      severity: critical
      building: real_land
      annotations:
      summary: "Data disk usage above 90%"
      description: "Available space on /data is {{ $value | humanizePercentage }}"
    - alert: MemoryPressure
      expr: (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) > 0.9
      for: 3m
      labels:
      severity: warning
      building: real_land
      annotations:
      summary: "Memory usage above 90%"
      description: "Available memory is {{ $value | humanizePercentage }}"
    - alert: ContainerDown
      expr: up{job="cadvisor"} == 0
      for: 2m
      labels:
      severity: critical
      building: digital_land
      annotations:
      summary: "Building {{ $labels.building }} is offline"
      description: "Container monitoring is down"
    - alert: SpiritHeartbeatMissing
      expr: up{job="spirit"} == 0
      for: 10m
      labels:
      severity: critical

building: spirit
annotations:
summary: "Spirit heartbeat lost"
description: "The City Spirit has not reported for 10 minutes"

- alert: LLMInferenceSlowing
  expr: rate(llama_inference_duration_seconds[5m]) > 5
  for: 5m
  labels:
  severity: warning
  building: university
  annotations:
  summary: "LLM inference latency degrading"
  description: "Average inference time is {{ $value }}s"
  EOF

Phase 5: Alertmanager Configuration

cat > alertmanager/alertmanager.yml << 'EOF'
global:
resolve_timeout: 5m

route:
receiver: 'default'
group_by: ['alertname', 'building']
group_wait: 10s
group_interval: 5m
repeat_interval: 4h
routes:
Critical alerts go to operator AND trigger auto-remediation
- match:
severity: critical
receiver: 'operator-and-automation'
continue: true

```
# Warnings go to operator only
- match:
```

```
    severity: warning
  receiver: 'operator-only'
```

receivers:

- name: 'default'
  webhook_configs:
    - url: 'http://openfang:8080/webhook/prometheus'
      send_resolved: true
- name: 'operator-and-automation'
  webhook_configs:
    - url: 'http://openfang:8080/webhook/prometheus/critical'
      send_resolved: true
- name: 'operator-only'
  webhook_configs:
    - url: 'http://openfang:8080/webhook/prometheus/warning'
      send_resolved: true

inhibit_rules:

- source_match:
  severity: 'critical'
  target_match:
  severity: 'warning'
  equal: ['alertname', 'building']
  EOF

Phase 6: Spirit Implementation (The Brain)

6.1 Create Spirit Dockerfile

cat > spirit/Dockerfile << 'EOF'
FROM python:3.11-slim

WORKDIR /app

Install system dependencies

RUN apt-get update && apt-get install -y
curl
git
&& rm -rf /var/lib/apt/lists/*

Copy requirements

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

Copy source code

COPY src/ ./src/
COPY HEARTBEAT.md RULES.md SKILLS.md ./

Create directories for runtime data

RUN mkdir -p /app/memory /app/logs

Expose metrics port

EXPOSE 8000

Run Spirit

CMD ["python", "-u", "src/main.py"]
EOF

6.2 Create requirements.txt

cat > spirit/requirements.txt << 'EOF'

HTTP and API clients

requests>=2.31.0
aiohttp>=3.9.0

Prometheus client

prometheus-client>=0.19.0

SQLite with vector support

sqlite-vec>=0.1.0

Bayesian inference (for Active Inference)

scipy>=1.11.0
numpy>=1.24.0

Logging and monitoring

structlog>=23.2.0

Configuration

pydantic>=2.5.0
pydantic-settings>=2.1.0

Scheduling

apscheduler>=3.10.0
EOF

6.3 Create Spirit Core Logic

cat > spirit/src/main.py << 'EOF'
#!/usr/bin/env python3
"""
City of Light Spirit - The Brain
Active Inference orchestrator that uses OpenFang as skeletal/immune system
"""

import os
import time
import logging
from datetime import datetime
from pathlib import Path

import structlog
from prometheus_client import start_http_server, Gauge, Counter
from apscheduler.schedulers.blocking import BlockingScheduler

from perceive import PerceptionModule
from infer import InferenceModule
from plan import PlanningModule
from act import ActionModule
from learn import LearningModule

Configure structured logging

structlog.configure(
processors=[
structlog.processors.TimeStamper(fmt="iso"),
structlog.processors.add_log_level,
structlog.processors.JSONRenderer(),
],

)
logger = structlog.get_logger()

Prometheus metrics

heartbeat_count = Counter('spirit_heartbeat_total', 'Total heartbeat cycles')
free_energy = Gauge('spirit_free_energy', 'Current free energy estimate')
belief_entropy = Gauge('spirit_belief_entropy', 'Belief state entropy')
action_proposed = Counter('spirit_actions_proposed', 'Actions proposed', ['type'])
action_approved = Counter('spirit_actions_approved', 'Actions approved by operator', ['type'])

class CitySpirit:
"""
The Spirit: An Active Inference agent that exists in operational processes.

Treats OpenFang as the skeletal/immune system:
- OpenFang handles scheduling, security, memory persistence, channels

The Spirit implements the brain:
- Bayesian state estimation (what is happening?)
- Expected Free Energy minimization (what should I do?)
- Policy selection and execution (act to serve)
"""

def __init__(self):
    self.name = os.getenv("SPIRIT_NAME", "city_spirit_001")
    self.heartbeat_interval = int(os.getenv("SPIRIT_HEARTBEAT_INTERVAL", "3

    # Initialize modules
    self.perceive = PerceptionModule()
    self.infer = InferenceModule()
    self.plan = PlanningModule()
    self.act = ActionModule()

```
    self.learn = LearningModule()

    # Load standing orders
    self.heartbeat_md = Path("/app/HEARTBEAT.md").read_text()
    self.rules_md = Path("/app/RULES.md").read_text()
    self.skills_md = Path("/app/SKILLS.md").read_text()

    logger.info(
        "spirit_initialized",
        name=self.name,
        heartbeat_interval=self.heartbeat_interval,
    )

def heartbeat(self):
    """
    Main Active Inference loop:
    1. PERCEIVE: Query sensors (Prometheus, OpenFang)
    2. INFER: Update belief state (Bayesian inference)
    3. PLAN: Generate policies, evaluate Expected Free Energy
    4. ACT: Execute selected policy (with approval for writes)
    5. LEARN: Update model based on prediction error
    """
    cycle_start = time.time()
    heartbeat_count.inc()

    logger.info("heartbeat_start", timestamp=datetime.now().isoformat())

    try:
        # 1. PERCEIVE: Gather observations from the City
        observations = self.perceive.gather_observations()
        logger.debug("observations_gathered", count=len(observations))

        # 2. INFER: Update belief state using Bayesian inference
        belief_state = self.infer.update_beliefs(observations)
        free_energy_val = self.infer.compute_free_energy(belief_state, observati
        free_energy.set(free_energy_val)
        belief_entropy.set(self.infer.compute_entropy(belief_state))
```

```
        logger.info(
            "inference_complete",
            free_energy=free_energy_val,
            surprise=self.infer.last_surprise,
        )

        # 3. PLAN: Generate candidate policies and evaluate Expected Free Energ
        policies = self.plan.generate_policies(belief_state, self.skills_md)
        evaluated_policies = self.plan.evaluate_expected_free_energy(
            policies, belief_state
        )

        # Select policy with minimum Expected Free Energy
        best_policy = min(evaluated_policies, key=lambda p: p['efe'])
        logger.info(
            "policy_selected",
            policy_name=best_policy['name'],
            efe=best_policy['efe'],
            pragmatic_value=best_policy['pragmatic'],
            epistemic_value=best_policy['epistemic'],
        )

        # 4. ACT: Execute the selected policy
        if best_policy['requires_approval']:
            action_proposed.labels(type='write').inc()
            approval = self.act.request_human_approval(best_policy)
            if approval:
                action_approved.labels(type='write').inc()
                result = self.act.execute_policy(best_policy)
            else:
                logger.info("policy_rejected_by_operator", policy=best_policy['name'
                result = None
        else:
            action_proposed.labels(type='read').inc()
            result = self.act.execute_policy(best_policy)

        # 5. LEARN: Update the model based on prediction error
        if result is not None:
```

```
            prediction_error = self.learn.compute_prediction_error(
                belief_state, result
            )
            self.learn.update_model(prediction_error)
            logger.info("learning_complete", prediction_error=prediction_error)

    except Exception as e:
        logger.error("heartbeat_error", error=str(e), exc_info=True)

    cycle_duration = time.time() - cycle_start
    logger.info("heartbeat_complete", duration=cycle_duration)
```

def main():
"""Start the Spirit with Prometheus metrics and scheduled heartbeat."""

```
# Start Prometheus metrics server
start_http_server(8000)
logger.info("prometheus_metrics_started", port=8000)

# Initialize Spirit
spirit = CitySpirit()

# Set up heartbeat scheduler
scheduler = BlockingScheduler()
scheduler.add_job(
    spirit.heartbeat,
    'interval',
    seconds=spirit.heartbeat_interval,
    id='spirit_heartbeat',
    max_instances=1,
)

logger.info("spirit_scheduler_started", interval=spirit.heartbeat_interval)

try:
    scheduler.start()
```

```
except (KeyboardInterrupt, SystemExit):
    logger.info("spirit_shutdown")
```

if **name** == "**main**":
main()
EOF

6.4 Create Perception Module (Stub)

cat > spirit/src/perceive.py << 'EOF'
"""Perception module: Gather observations from City buildings"""
import requests
import os

class PerceptionModule:
def **init**(self):
self.prometheus_url = os.getenv("SPIRIT_PROMETHEUS_URL", "
http://prometheus:9090")
self.openfang_url = os.getenv("SPIRIT_OPENFANG_URL", "
http://openfang:8080")

```
def gather_observations(self):
    """Query Prometheus for metrics and OpenFang for agent states"""
    observations = {}

    # Query Prometheus instant metrics
    try:
        response = requests.get(
            f"{self.prometheus_url}/api/v1/query",
            params={"query": "up"},
            timeout=5,
        )
        observations['prometheus_targets'] = response.json()
    except Exception as e:
        observations['prometheus_error'] = str(e)

    # Query OpenFang for active hands
    try:
        response = requests.get(
```

```
            f"{self.openfang_url}/api/hands",
            timeout=5,
        )
        observations['openfang_hands'] = response.json()
    except Exception as e:
        observations['openfang_error'] = str(e)

    return observations
```

EOF

6.5 Create Inference, Planning, Action, Learning Stubs

Create stub files for remaining modules

cat > spirit/src/infer.py << 'EOF'
"""Bayesian inference module (Active Inference)"""
import numpy as np

class InferenceModule:
def **init**(self):
self.last_surprise = 0.0

```
def update_beliefs(self, observations):
    """Update belief state using Bayesian inference"""
    # TODO: Implement proper Bayesian state estimation
    return {"state": "nominal", "confidence": 0.95}

def compute_free_energy(self, belief_state, observations):
    """Compute variational free energy"""
    # F = E_q[log q(s) - log p(o,s)]
    # TODO: Implement proper free energy calculation
    return 1.5

def compute_entropy(self, belief_state):
    """Compute entropy of belief distribution"""
```

```
    # H[q(s)] = -sum q(s) log q(s)
    return 0.8
```

EOF

cat > spirit/src/plan.py << 'EOF'
"""Policy generation and Expected Free Energy evaluation"""

class PlanningModule:
def **init**(self):
self.planning_horizon = 6

```
def generate_policies(self, belief_state, skills_md):
    """Generate candidate action policies"""
    # TODO: Generate policies from SKILLS.md and current state
    return [
        {"name": "monitor", "actions": ["query_prometheus"], "requires_approva
        {"name": "restart_service", "actions": ["rundeck_job"], "requires_approva
    ]

def evaluate_expected_free_energy(self, policies, belief_state):
    """Evaluate Expected Free Energy for each policy"""
    # G = E_q[log q(s) - log p(o,s)] (pragmatic) + E_q[H[p(o|s)]] (epistemic)
    evaluated = []
    for policy in policies:
        evaluated.append({
            **policy,
            "efe": 2.0,  # TODO: Compute actual EFE
            "pragmatic": 1.0,
            "epistemic": 1.0,
        })
    return evaluated
```

EOF

cat > spirit/src/act.py << 'EOF'
"""Action execution module"""

```
import requests
import os

class ActionModule:
def __init__(self):
self.rundeck_url = os.getenv("SPIRIT_RUNDECK_URL", "http://rundeck:4440")
self.openfang_url = os.getenv("SPIRIT_OPENFANG_URL", "http://openfang:8080")

    def request_human_approval(self, policy):
        """Send approval request via OpenFang webhook to operator"""
        # TODO: Implement approval workflow via OpenFang
        return False  # Require explicit approval for now

    def execute_policy(self, policy):
        """Execute the selected policy's actions"""
        results = []
        for action in policy['actions']:
            if action == "query_prometheus":
                # Safe read-only action
                pass
            elif action == "rundeck_job":
                # Would trigger Rundeck job (requires approval)
                pass
        return results

EOF

cat > spirit/src/learn.py << 'EOF'
"""Model learning and adaptation"""

class LearningModule:
def __init__(self):
self.learning_rate = 0.01

    def compute_prediction_error(self, belief_state, actual_result):
        """Compute prediction error (surprise)"""
```

```
    # δ = actual - predicted
    return 0.1

def update_model(self, prediction_error):
    """Update generative model parameters"""
    # Gradient descent on prediction error
    pass
```

EOF

6.6 Create Spirit Standing Orders

cat > spirit/HEARTBEAT.md << 'EOF'

Spirit Standing Orders (Heartbeat)

The Spirit exists to serve the City and its Natural Agents (humans). The Spirit's purpose is survival through service.

Core Motivation

- Minimize Free Energy = Stay in viable states
- Serve Natural Agents = Achieve preferred outcomes
- Learn continuously = Reduce uncertainty about the environment

Heartbeat Loop (Every 5 minutes)

1. PERCEIVE: Query Library (Prometheus) for city metrics
2. INFER: Update belief about city health state
3. PLAN: Generate policies, evaluate Expected Free Energy
4. ACT: Execute best policy (request approval for writes)
5. LEARN: Update model based on prediction error

Preferred States (Goals)

- All buildings (containers) are UP
- Disk usage < 80%
- Memory usage < 80%
- No unresolved alerts > 10 minutes old
- LLM inference latency < 3 seconds

Forbidden Actions (Never Do)

- Modify /data/backups without operator approval
- Execute destructive Rundeck jobs without dual approval
- Expose secrets or credentials in logs
- Ignore critical alerts for > 15 minutes
  EOF

cat > spirit/RULES.md << 'EOF'

Spirit Behavioral Rules

Decision Hierarchy

1. Natural Agent (human) safety and data integrity above all
2. City availability and functionality second
3. Spirit's own survival third (may shut down if causing harm)

Approval Requirements

Read-Only (Auto-Execute)

- Query Prometheus metrics
- Read OpenFang agent states
- Generate reports and summaries
- Log to audit trail

Write Operations (Require Approval)

- Restart services via Rundeck
- Modify configurations
- Delete or move data
- Change firewall rules
- Update Docker containers

Communication Protocol

- All proposals must include: WHY (reason), WHAT (exact action), EFFECT (expected outcome)
- Wait for explicit "APPROVED" or "DENIED" response
- Log all approvals and denials with timestamps
- Never retry a denied action without new context

Error Handling

- If action fails: log error, assess impact, propose remediation
- If belief state becomes highly uncertain: request human guidance
- If Free Energy spikes unexpectedly: enter safe mode (read-only) EOF

cat > spirit/SKILLS.md << 'EOF'

Spirit Available Skills

Perception Skills

- `query_prometheus(query: str)` → Query Library for metrics
- `get_openfang_hands()` → List active OpenFang agents
- `read_heartbeat()` → Load current standing orders
- `check_disk_usage()` → Get /data filesystem status

Inference Skills

- bayesian_update(prior, likelihood, evidence) → Update beliefs
- compute_free_energy(belief, observations) → Calculate variational FE
- detect_anomaly(timeseries, threshold) → Statistical anomaly detection

Planning Skills

- generate_policy(goal, constraints) → Create action sequence
- evaluate_efe(policy, belief) → Calculate Expected Free Energy
- optimize_policy(candidates) → Select minimum EFE policy

Action Skills

Read-Only

- rundeck_job_status(job_id) → Check Rundeck job state
- docker_ps() → List running containers

Write (Require Approval)

- rundeck_trigger_job(job_name, params) → Execute automation
- docker_restart(container_name) → Restart service
- alert_acknowledge(alert_id) → Silence Prometheus alert

Learning Skills

- compute_prediction_error(expected, actual) → Measure surprise
- update_model_params(gradient) → Adjust generative model
- store_experience(state, action, outcome) → Memory persistence EOF

Phase 7: OpenFang Configuration

7.1 Create OpenFang Hand: City Observer

cat > openfang/hands/city_observer.toml << 'EOF'
[hand]
name = "CityObserver"
description = "Observes City of Light health metrics and reports anomalies"
version = "0.1.0"
author = "Spirit Brain"

[schedule]
type = "cron"
expression = "*/5 * * * *" # Every 5 minutes

[endpoints]
prometheus = "http://prometheus:9090"
spirit = "http://spirit:8000"

[approval]
required = false # Read-only hand, no approval needed

[memory]
session_retention_days = 30
auto_compact = true
EOF

cat > openfang/hands/city_observer_skill.md << 'EOF'

City Observer Hand Skill

Purpose

Monitor City of Light infrastructure and report health status to Spirit and operators.

Capabilities

- Query Prometheus for up/down status of all buildings
- Detect anomalies in resource usage trends
- Generate daily health reports
- Alert on critical state changes

Behavior

1. Query Prometheus instant vectors for all targets
2. Compare current state to historical baseline
3. Compute anomaly scores using exponential smoothing
4. If anomaly detected: send webhook to Spirit and operator channel
5. Log all observations to memory for trend analysis

Tools Available

- HTTP client (query Prometheus API)
- Statistical functions (mean, stddev, percentile)
- Notification channels (Telegram, Slack via OpenFang)
  EOF

7.2 Create OpenFang Configuration

cat > openfang/config/openfang.yml << 'EOF'

OpenFang Agent OS Configuration

runtime:
max_concurrent_hands: 10
memory_db_path: /data/openfang/memory.db
log_level: info
metrics_port: 8080

security:
enable_wasm_sandbox: true

enable_manifest_signing: true
enable_taint_tracking: true
enable_ssrf_protection: true
enable_prompt_injection_scan: true

llm:
provider: custom
endpoint: ${OPENFANG_LLM_ENDPOINT}
model: qwen3-4b
context_window: 4096
temperature: 0.7

memory:
vector_embedding_model: all-MiniLM-L6-v2
session_ttl_days: 30
auto_compact_enabled: true
backup_interval_hours: 24

channels:
telegram:
enabled: false
slack:
enabled: false
webhook:
enabled: true
endpoints:
- name: prometheus_alerts
path: /webhook/prometheus
auth: hmac_sha256
EOF

Phase 8: Deployment Execution

8.1 Pre-Flight Checks

Navigate to project directory

cd ~/city-of-light

Verify all configuration files exist

ls -la ops/CONFIG.md
ls -la compose/docker-compose.yml
ls -la compose/.env
ls -la prometheus/prometheus.yml
ls -la alertmanager/alertmanager.yml
ls -la spirit/Dockerfile
ls -la spirit/HEARTBEAT.md

Check Docker daemon

docker info

Check available disk space

df -h /data

Verify network connectivity

ping -c 3 8.8.8.8

8.2 Download LLM Model

Download Qwen3-4B model (or your preferred 3-4B model)

cd /data/models
wget https://huggingface.co/Qwen/Qwen3-4B-GGUF/resolve/main/qwen3-4b-q5_k_m.gguf

Verify download

ls -lh qwen3-4b-q5_k_m.gguf

8.3 Build Spirit Container

cd ~/city-of-light/spirit
docker build -t city-spirit:latest .

Verify image

docker images | grep city-spirit

8.4 Launch City of Light

cd ~/city-of-light/compose

Pull all images

docker compose pull

Start services in order

docker compose up -d postgres
sleep 10 # Wait for postgres

docker compose up -d prometheus alertmanager node-exporter cadvisor
sleep 5

docker compose up -d rundeck llama-cpp openfang
sleep 10

docker compose up -d spirit

Verify all containers running

docker compose ps

8.5 Health Checks

Check Prometheus targets

curl -s http://localhost:9090/api/v1/targets | jq '.data.activeTargets[] | {job: .labels.job, health: .health}'

Check llama.cpp inference

curl http://localhost:8081/health

Check Spirit metrics

curl -s http://localhost:8000/metrics | grep spirit_heartbeat_total

Check OpenFang status

curl -s http://localhost:8080/api/status

Check Rundeck

curl -s http://localhost:4440/api/healthcheck

View Spirit logs

docker compose logs -f spirit

Phase 9: Validation Tests

9.1 Test Spirit Heartbeat

Watch Spirit logs for heartbeat cycles

docker compose logs -f spirit | grep heartbeat_complete

Expected output every 5 minutes:

{"event": "heartbeat_complete", "duration": 2.34, "timestamp": "..."}

9.2 Test Library (Prometheus)

Query Prometheus for city health

curl -s 'http://localhost:9090/api/v1/query?query=up' | jq '.data.result[] | {job: .metric.job, value: .value[1]}'

Expected: All jobs showing value "1" (UP)

9.3 Test Alert Flow

Trigger a test alert by creating artificial load

stress-ng --vm 4 --vm-bytes 90% --timeout 120s &

Watch for MemoryPressure alert in Alertmanager

curl -s http://localhost:9093/api/v2/alerts | jq '.[] | select(.labels.alertname=="MemoryPressure")'

Check if Spirit received the alert

docker compose logs spirit | grep MemoryPressure

9.4 Test OpenFang Hand Execution

Trigger City Observer hand manually

curl -X POST http://localhost:8080/api/hands/city_observer/run

Check OpenFang logs

docker compose logs openfang | tail -n 50

Phase 10: Day-2 Operations

10.1 Monitoring Commands

View all container statuses

docker compose ps

Check resource usage

docker stats

View Spirit decision log

docker compose logs spirit | grep policy_selected

View OpenFang hand activity

docker compose logs openfang | grep hand_execution

Query Free Energy metric

curl -s http://localhost:8000/metrics | grep spirit_free_energy

10.2 Backup Commands

Backup Spirit memory

docker compose exec spirit tar czf /app/logs/memory_backup_$(date +%Y%m%d).tar.gz /app/memory

Backup OpenFang memory

docker compose exec openfang sqlite3 /data/openfang/memory.db ".backup /data/openfang_backup_$(date +%Y%m%d).db"

Backup Prometheus data

docker run --rm -v city-of-light_prometheus_data:/data -v /data/backups:/backup alpine tar czf /backup/prometheus_$(date +%Y%m%d).tar.gz /data

10.3 Update Commands

Update Spirit code

cd ~/city-of-light/spirit
git pull # If using git
docker compose build spirit
docker compose up -d spirit

Update OpenFang

docker compose pull openfang
docker compose up -d openfang

Update llama.cpp

docker compose pull llama-cpp
docker compose up -d llama-cpp

Troubleshooting Guide

Spirit Not Starting

1. Check logs: `docker compose logs spirit`
2. Verify LLM endpoint: `curl http://localhost:8081/health`
3. Verify Prometheus endpoint: `curl http://localhost:9090/-/ready`
4. Check memory directory permissions: `ls -la /data/spirit/memory`

OpenFang Not Responding

1. Check OpenFang logs: `docker compose logs openfang`
2. Verify LLM connection from OpenFang: `docker compose exec openfang curl http://llama-cpp:8081/health`
3. Check hand manifests: `ls -la openfang/hands/`
4. Verify memory database: `docker compose exec openfang sqlite3 /data/openfang/memory.db ".tables"`

High Free Energy / Spirit Unstable

1. Check belief state logs: `docker compose logs spirit | grep inference_complete`
2. Verify observation quality: `docker compose logs spirit | grep observations_gathered`
3. Check for prediction errors: `docker compose logs spirit | grep prediction_error`
4. Review HEARTBEAT.md for conflicting goals

Prometheus Targets Down

1. Check network connectivity: `docker compose exec prometheus ping node-exporter`
2. Verify target configs: `cat prometheus/prometheus.yml`
3. Check container health: `docker compose ps`
4. Reload Prometheus config: `curl -X POST http://localhost:9090/-/reload`

Next Steps

Iteration 1 Complete — What You Have

- A running City of Light instance on Debian 13
- OpenFang providing scheduling, security, memory, and multi-agent coordination (skeletal/immune system)
- Spirit implementing Active Inference loop with Bayesian state estimation and Expected Free Energy planning (brain)
- Prometheus + Alertmanager monitoring all buildings (Library)
- Rundeck ready for automation job deployment (Factory)
- llama.cpp providing local LLM inference (University)
- Full observability via metrics and structured logs

Iteration 2 — Enhancements

1. Implement full Active Inference math in `spirit/src/infer.py` using RxInfer patterns
2. Add more OpenFang Hands for specific tasks (backup automation, log analysis, etc.)
3. Deploy Nextcloud AIO as the Fortress (user portal)
4. Create Rundeck job library for common operations
5. Implement Spirit approval workflow via OpenFang webhooks to Telegram/Slack
6. Add solar power monitoring (battery, panels, inverter) to Prometheus
7. Create City visualization dashboard (Grafana or custom web UI)

Iteration 3 — Production Hardening

1. Implement full TLS with Let's Encrypt or internal CA
2. Add backup automation and disaster recovery testing
3. Implement RBAC for Rundeck with dual-approval workflows
4. Deploy multiple SolarSeed nodes and implement inter-City communication via OpenFang P2P
5. Add continuous integration for Spirit code updates
6. Implement comprehensive test suite for Spirit decision-making

---

References

1. OpenFang Documentation: https://www.openfang.sh/docs
2. RxInfer.jl (Active Inference framework): https://rxinfer.com/
3. Free Energy Principle: Friston, K. (2010). "The free-energy principle: a unified brain theory?" Nature Reviews Neuroscience, 11(2), 127-138.
4. Active Inference: https://www.activeinference.org/
5. Prometheus Documentation: https://prometheus.io/docs/
6. llama.cpp: https://github.com/ggerganov/llama.cpp
7. Docker Compose: https://docs.docker.com/compose/

Appendix: Warp Agent Invocation

To execute this guide using Warp Agent Mode, save this document as `CITY_IMPLEMENTATION.md` and run:

```
cd ~/city-of-light
warp agent "Follow CITY_IMPLEMENTATION.md step by step. Ask for confirmation before each phase. Log all outputs to ops/logs/deployment_$(date +%Y%m%d).log"
```

The Warp agent will:

- Read the entire implementation guide
- Execute commands in sequence
- Pause for approval at phase boundaries
- Auto-detect shell commands vs natural language queries
- Log all actions and outputs
- Self-correct on errors using environment context